Institut Universitaire de Technologie
du Limousin

R3.04 – Qualité de développement

Les design patterns
Adapter et Façade

Réalisé par :
Deneuve Trystan
Diang Seydi
Oscar

BUT Informatique – Semestre 3
IUT du Limousin

Sommaire

1. Les design patterns et leurs familles **| 3**

2. La problématique : incompatibilité et complexité d'accès **| 3**

3. Les principes des patterns Adapter et Façade **| 3**

4. Les participants des deux patterns **| 4**

5. Exemples d'utilisation métier **| 4**

6. Analyse des principes SOLID et limites **| 5**

7. Conclusion et références **| 5**

1. Les design patterns et leurs familles

Un design pattern (ou patron de conception) est une solution éprouvée et standardisée à un problème récurrent de conception logicielle orientée objet. Publiés à l'origine en 1994 par le « Gang of Four » (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides), ils constituent un vocabulaire commun et universel facilitant la collaboration et la maintenance des projets.

Le GoF répartit les 23 patrons majeurs en trois grandes familles :

- **Les patrons de création :** gèrent l'instanciation des objets en isolant la logique de fabrication (ex. Singleton, Factory, Builder).
- **Les patrons de structure :** organisent l'agencement et la composition des classes et des objets pour former des structures plus vastes et flexibles (ex. Adapter, Façade, Decorator, Composite).
- **Les patrons de comportement :** définissent les mécanismes de communication, de délégation et de répartition des responsabilités entre objets (ex. Stratégie, Observateur, Commande, État).

Adapter et Façade sont deux patrons structurels majeurs. Ils interviennent tous deux sur des classes existantes sans devoir en modifier le code source.

2. La problématique : incompatibilité et complexité d'accès

Deux freins récurrents compromettent la maintenabilité des applications d'entreprise :

- **L'incompatibilité d'interface :** Une classe cliente attend une signature ou un contrat précis (ex. une interface Paiement avec payer(double euros)), mais la bibliothèque tierce ou le module externe non modifiable fournit une signature divergente (ex. debiter(long centimes)). Coupler directement le client à la classe concrète pollue la logique métier avec des détails de conversion technique et interdit les tests unitaires isolés.
- **L'enchevêtrement des sous-systèmes :** Pour satisfaire une intention métier simple (comme valider une commande ou fermer une caisse), le code appelant est contraint d'orchestrer manuellement une dizaine de composants distincts (Stock, Paiement, Livraison, Facturation, Archive). Chaque client doit mémoriser l'ordre exact des opérations, générant une duplication massive de code d'orchestration et un couplage excessif.

3. Les principes des patterns Adapter et Façade

Le pattern Adapter : Son intention est de convertir l'interface d'une classe existante en l'interface attendue par le client. Il agit comme un traducteur ou une passerelle (à l'image d'un adaptateur de prise électrique français/anglais). Le client n'interagit qu'avec sa cible familière ; l'adaptateur traduit les requêtes, convertit les types et les unités, puis délègue le traitement réel à l'objet adapté sans que ni l'un ni l'autre ne soient modifiés.

Le pattern Façade : Son intention est de fournir une interface unique et unifiée devant un ensemble complexe de classes formant un sous-système. La façade agit comme un guichet unique ou un majordome : elle regroupe et ordonnance les appels verbeux du sous-système derrière une seule méthode claire (ex. passerCommande()), tout en laissant le sous-système accessible en direct pour les besoins experts.

4. Les participants des deux patterns

La littérature du GoF identifie formellement les acteurs suivants pour chaque patron :

Participants du pattern Adapter (Structure 1 vers 1) :

Rôle générique (GoF)	Responsabilité & Description
Client	Utilise uniquement l'interface Cible. Il ignore totalement l'existence de l'Adapté et de l'Adaptateur.
Cible (Target)	Interface ou contrat formel attendu et manipulé par le client métier.
Adaptateur (Adapter)	Implémente la Cible et détient une référence vers l'Adapté. Il traduit les paramètres, unités et exceptions.
Adapté (Adaptee)	Classe existante ou bibliothèque externe fournissant le service réel, mais dont l'interface est incompatible.

Participants du pattern Façade (Structure 1 vers N) :

Rôle générique (GoF)	Responsabilité & Description
Façade (Facade)	Offre une interface de haut niveau simple et unifiée. Orchestre et délègue les requêtes aux classes appropriées.
Sous-système (Subsystem)	Ensemble hétérogène de classes réalisant le travail métier effectif. Elles n'ont aucune connaissance de la façade.
Client	Communique uniquement avec la Façade pour accomplir ses scénarios sans devoir instancier ni coordonner le sous-système.

5. Exemples d'utilisation métier

Cas 1 – Paiement bancaire e-commerce (Adapter) : Notre ServiceCommande manipule l'interface Paiement { boolean payer(double euros, String carte); }. Le prestataire nous fournit un JAR fermé ApiBancaireExterne { int debiter(long centimes, String carte); }. La classe BanqueAdapter convertit les euros en centimes, invoque debiter(), et traduit le code retour 0 en booléen true.

Cas 2 – Clavier et manette dans un jeu vidéo (Adapter) : La classe Personnage attend un contrat Controle { double direction(); boolean saut(); }. Une manette Bluetooth renvoie un axe analogique brut entier entre -32768 et 32767 via axeX(). La classe ManetteAdapter effectue la mise à l'échelle vers l'intervalle [-1.0, 1.0] sans modifier le personnage.

6. Analyse des principes SOLID et limites

La robustesse des patrons structurels s'évalue au regard des principes SOLID :

S – Responsabilité Unique (SRP)	Adapter : Isole la tuyauterie technique (conversions, formats) en dehors de la logique métier. Façade : Regroupe l'orchestration, mais attention au risque de dériver vers une classe-dieu (God Object).
O – Ouvert / Fermé (OCP)	Adapter : Pleinement respecté. Ajouter une nouvelle banque consiste à créer une nouvelle classe sans rouvrir le service. Façade : En tension ; l'ajout d'une étape métier contraint souvent à modifier le corps de la façade existante.
L – Substitution de Liskov (LSP)	Adapter : Tout adaptateur se substitue parfaitement à l'interface cible sans altérer le comportement global attendu. Façade : Non concerné s'il n'y a pas d'héritage direct.
I – Ségrégation d'Interfaces (ISP)	Adapter : L'interface cible reste mince et ajustée aux besoins stricts du client. Façade : Risque d'exposer une interface trop large si elle accumule trop de variantes de cas d'usage.
D – Inversion des Dépendances (DIP)	Adapter & Façade : Le domaine métier dépend exclusivement d'abstractions de haut niveau (ports) plutôt que de bibliothèques tierces concrètes.

Limites des patterns :

• Adapter multiplie le nombre de classes intermédiaires. Si l'on a la maîtrise complète du code source des deux côtés, un refactoring direct des interfaces est souvent préférable à une surcouche d'adaptation.

• Façade masque la granularité interne. Elle peut restreindre l'accès à certaines options avancées du sous-système si elle est trop rigide, ou au contraire devenir un goulot d'étranglement monolithique.

7. Conclusion et références

Adapter et Façade sont deux réponses complémentaires à la complexité structurelle des applications logicielles. Adapter résout un problème d'incompatibilité de contrat (1 vers 1) en s'alignant sur une interface existante, tandis que Façade résout un problème de surcharge cognitive et de couplage (1 vers N) en créant une nouvelle interface simplifiée. Dans l'architecture hexagonale (Ports & Adapters), ces patrons constituent la clé de voûte permettant d'isoler le cœur métier des contingences technologiques.

Références bibliographiques du support :

- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- Eric Freeman, Elisabeth Robson. Head First Design Patterns (2e éd.). O'Reilly Media, 2020.
- Alistair Cockburn. Hexagonal Architecture (Ports and Adapters), 2005.
- Refactoring.Guru. Design Patterns : « Adapter » et « Façade » (https://refactoring.guru/fr).
- Support de cours R3.04 – Qualité de développement (BUT Informatique, IUT du Limousin).